

COMP 6231: Distributed Systems

Assignment 1

Architectural Design and Trade-Off Analysis of a Real-Time Ride-Sharing Platform Backend

Author: Travis Holt
Student ID: 40351596
Instructor: Dr. Reza Mirsalari
Date: June 30, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	System Requirements	2
1.2.1	Functional Requirements	2
1.2.2	Non-Functional Requirements	2
1.3	Report Organization	2
2	System Architecture Overview	3
3	Database Sharding	5
3.1	Sharding Strategy Comparison	5
3.2	Chosen Approach and Justification	5
4	Replication and Consistency Models	6
4.1	Consistency Model Comparison	6
4.2	Chosen Approach and Justification	6
5	Load Balancing	7
5.1	Layer 4 vs. Layer 7 Comparison	7
5.2	Chosen Approach and Justification	7
6	Trade-Off Analysis	8
7	Evaluation	9
8	Technology Recommendations	9

1 Introduction

1.1 Problem Statement

"...architect the backend infrastructure of a real-time ride-sharing platform similar to Uber or Lyft, using principles from distributed systems. Your task is to analyze and design a distributed backend system that supports high availability, low latency, and data consistency under high concurrency."

You must integrate and analyze the following core architectural components

- Database Sharding (Hash-based, Range-based, or Consistent Hashing)
- Replication and Consistency Models (Eventual, Strong, Quorum-based)
- Load Balancing (Layer 4 vs. Layer 7)

1.2 System Requirements

1.2.1 Functional Requirements

- Drivers broadcast live location and availability.
- Riders send trip requests that are matched with nearby drivers.
- The system supports live matching, route calculation, and payment processing.
- Data includes user accounts, trip histories, real-time GPS data, and pricing rules.

1.2.2 Non-Functional Requirements

- Global scale support (multi-region architecture assumed).
- Sub-200ms latency for ride-matching response in 90% of requests.
- High read/write throughput for hotspots (e.g., downtown during peak hours).

1.3 Report Organization

The proposed system is introduced graphically with a series of architecture diagrams with increasing granularity. Design considerations, trade-offs, and decisions are discussed subsequently. The report concludes with technology recommendations for implementing this system.

2 System Architecture Overview

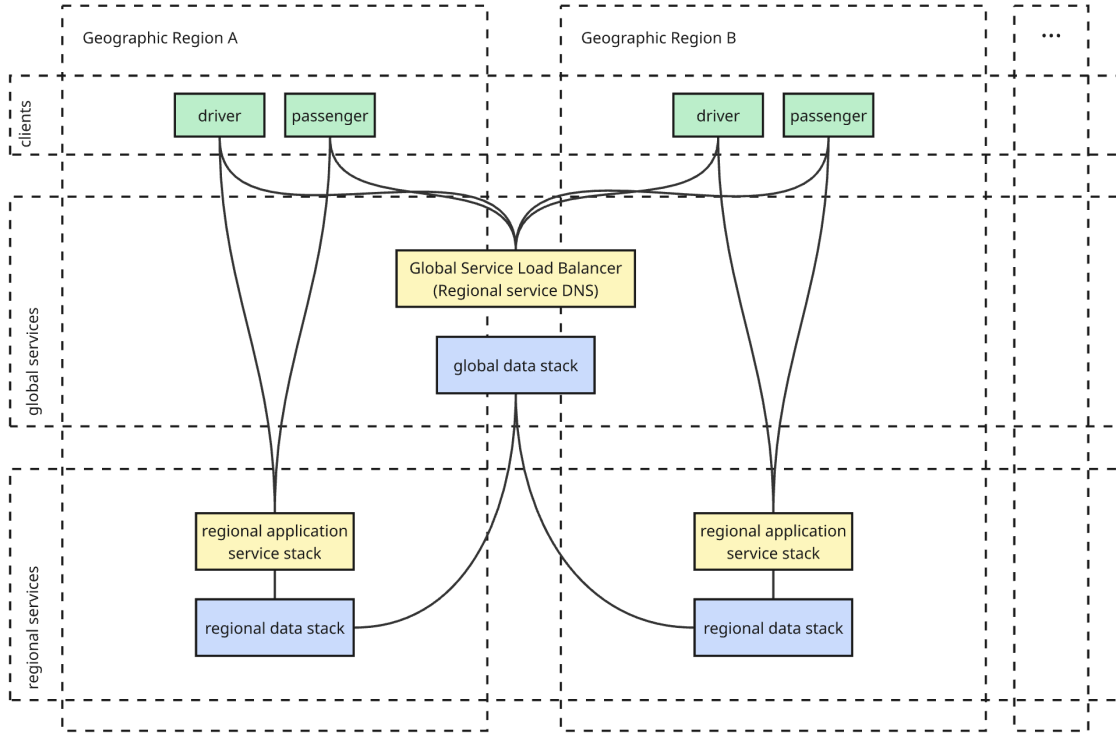


Figure 1: High-level system architecture.

At the highest level the system components can be categorized into three groups: clients, global services, and regional services. The personas of the clients may be either passengers or drivers and their interfaces may be native mobile applications or web browsers. The global services are responsible for authentication, authorization, account management, and regional service routing. The bulk of core functionality is provided by the regional services, of which there can be an arbitrary number of instances.

When clients begin interacting with the system they query the global service load balancer (GSLB) for determine the appropriate regional service endpoint with which to communicate. The GSLB's DNS resolution is governed by a geolocation routing policy that maps a client's current IP address to a regional service IP address. This policy may also consider availability, capacity, and load.

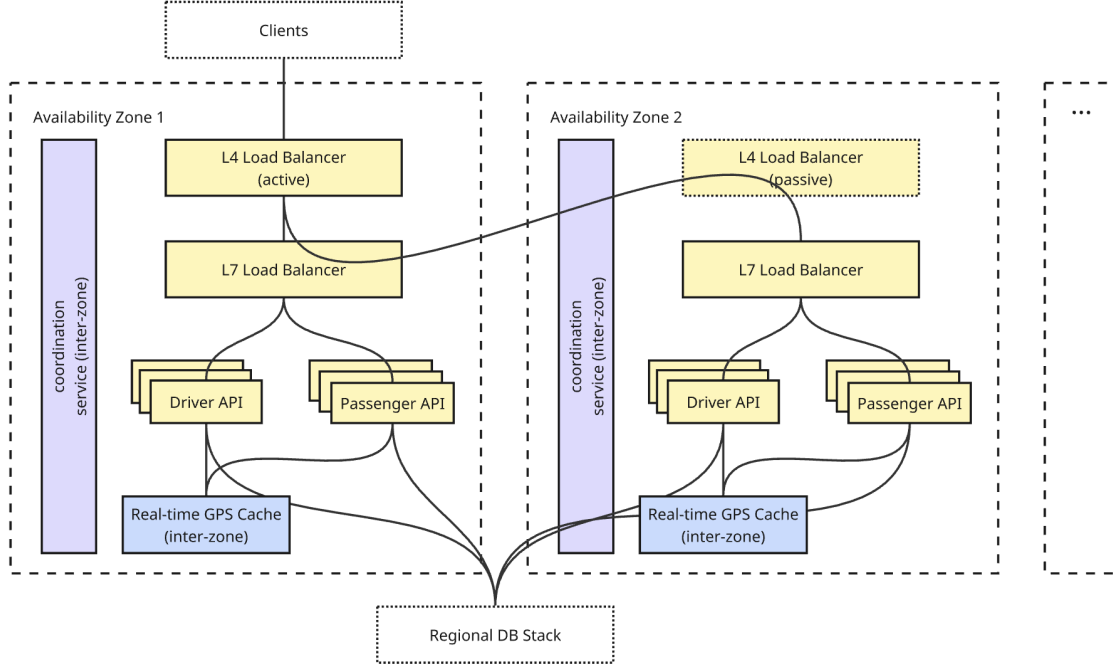


Figure 2: Regional application service stack architecture.

Once a client has been routed to a regional service endpoint a layer 4 load balancer (L4LB) proxies their requests to an availability zone (AZ). The purpose of the L4LB is to distribute load across AZs and adapt to outages in AZs. At any given time there is a single active L4LB in a region, backed by several passive instances that can take over should the active instance fail. Next, a layer 7 load balancer (L7LB) proxies requests to an appropriate application API based on the request path and headers. The L7LB is also responsible for TLS termination and rate limiting. This allows components deeper in the stack to focus on core functionality rather than security and load management.

Each application API has multiple instances running, with the specific number scaling with the current or anticipated load. The API services are stateless which allows them to be regarded as ephemeral. The diagram depicts two specific APIs for illustration purposes, but in reality there will likely be many more along with additional supporting microservices. Each region also has an inter-zone distributed cache for rapidly changing non-durable data such as GPS locations of users. Durable data is managed by regional data stacks. Finally, each region has a coordination service (CS) that spans AZs. The CS is responsible for leader election in the case of the L4LBs. It also provides service registration and discovery which enables the L7LB to route to a flexible pool of API instances.

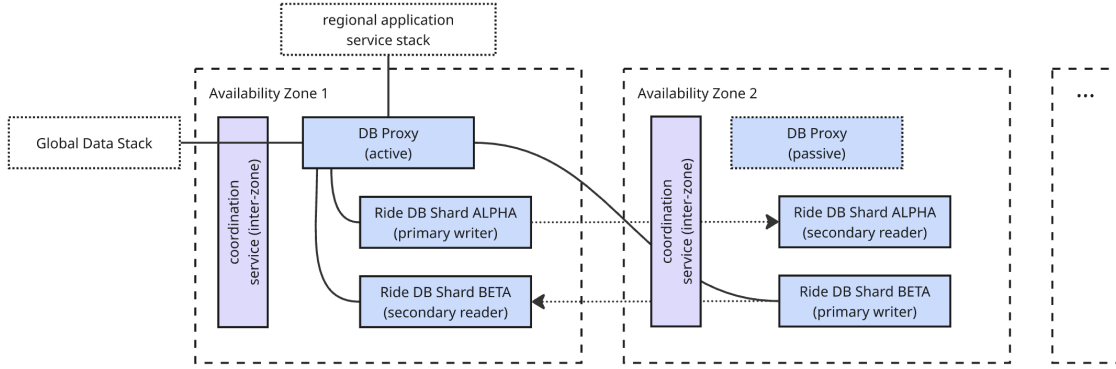


Figure 3: Regional data stack architecture.

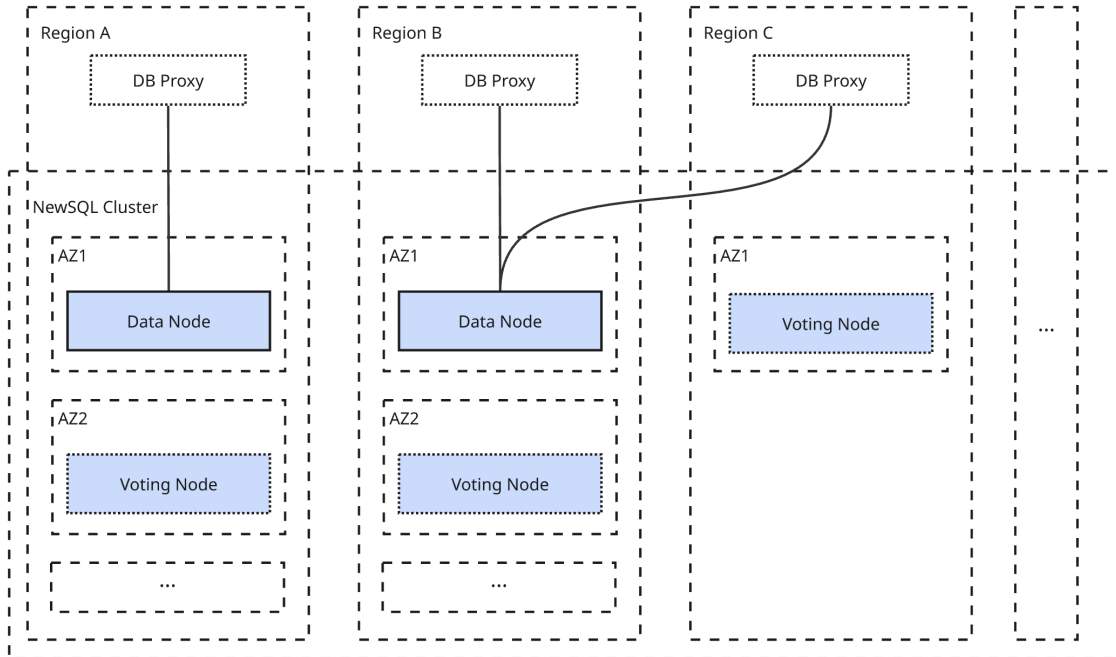


Figure 4: Global data stack architecture.

3 Database Sharding

3.1 Sharding Strategy Comparison

3.2 Chosen Approach and Justification

4 Replication and Consistency Models

4.1 Consistency Model Comparison

4.2 Chosen Approach and Justification

5 Load Balancing

5.1 Layer 4 vs. Layer 7 Comparison

5.2 Chosen Approach and Justification

6 Trade-Off Analysis

Table 1: Summary of architectural trade-offs.

Component	Decision	Key Trade-Off
Sharding		
Replication/Consistency		
Load Balancing		

7 Evaluation

CAP

Capacity

Latency

8 Technology Recommendations

Cloud provider: AWS, GCP, Azure

Provisioning: Kubernetes, Docker, Terraform, Ansible

Components: Kafka, Redis, PostgreSQL, MongoDB, Nginx, Envoy